



MANUAL DE ENSAMBLAJE E INSTALACIÓN

Checador Wi-Fi Skilled

Raspberry Pi y ESP32-S3 | Huella, código, operación offline y sincronización con RH

Versión técnica V85 · Agosto 2026

Objetivo

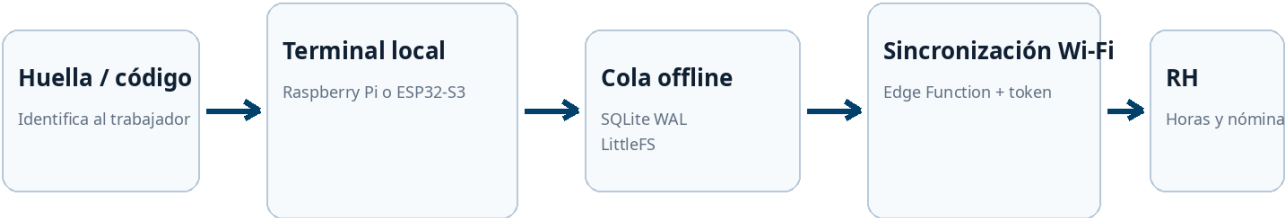
Construir un checador físico que registre entradas y salidas por huella o número de trabajador, continúe operando sin Internet y sincronice los eventos con RH cuando vuelva la conexión.

Documento preparado para Skilled Proyectos Industriales.

1. Qué se va a construir

El Checador Wi-Fi funciona como una terminal física independiente del navegador del CRM. La identificación ocurre en el equipo; las checadass se guardan primero de forma local y después se envían al backend del CRM mediante una credencial propia del dispositivo.

Flujo del Checador Wi-Fi Skilled



Si se pierde Internet, el evento permanece local y se envía cuando vuelve la conexión.

Arquitectura prevista en el código V85: Raspberry Pi usa SQLite en modo WAL; ESP32-S3 usa una cola NDJSON en LittleFS. Ambos envían a la función checador-sync con código y token de terminal.

Recomendación para Skilled

Usar Raspberry Pi como terminal principal si llevará pantalla táctil, administración local, diagnósticos y una futura cámara para reconocimiento facial. Usar ESP32-S3 cuando se quiera un lector compacto, económico y dedicado principalmente a huella/código.

Comparación rápida

Criterio	Raspberry Pi	ESP32-S3
Interfaz local	Muy cómoda: navegador, pantalla táctil y servicio Linux.	Más simple; requiere programar pantalla o botones.
Cola sin Internet	SQLite WAL, fácil de auditar.	LittleFS; suficiente para eventos compactos.
Huella	UART + Python + Adafruit Fingerprint.	UART + librería Adafruit Fingerprint.
Mantenimiento	SSH, logs, systemd, archivos de configuración.	Recompilar o usar consola serie para cambios.
Futuro rostro/cámara	Más apropiado.	No es la opción recomendada para visión.
Costo/consumo	Mayor.	Menor.

2. Materiales necesarios

Opción A - Raspberry Pi, recomendada

Elemento	Recomendación	Función
Raspberry Pi	Pi 4B 2 GB o superior; Pi 5 si después se añadirá visión.	Ejecuta el servicio, UI, base local y sincronización.
microSD	32 GB o más, de buena calidad.	Sistema operativo y base local.
Fuente	Fuente oficial/estable para el modelo elegido.	Evita reinicios y corrupción.
Pantalla	Táctil HDMI/DSI de 5 a 7 pulgadas, opcional pero recomendada.	Muestra estado, código y confirmaciones.
Lector de huella	UART/TTL compatible con el protocolo de Adafruit Fingerprint.	Captura y busca plantillas localmente.
Carcasa	Impresa o comercial, con ventilación y acceso al sensor.	Protección física.
Cables	Dupont o arnés asegurado; GND común.	UART y alimentación.
Buzzer/LED	Opcional.	Confirmación visual/sonora.

Opción B - ESP32-S3

Elemento	Recomendación	Función
ESP32-S3	DevKitC-1 o equivalente compatible con PlatformIO.	Control principal y Wi-Fi.
Lector de huella	UART/TTL compatible.	Identificación biométrica.
Fuente	USB 5 V estable para la placa.	Alimentación.
Pantalla	OLED/TFT opcional.	Estado e instrucciones.
Carcasa	Compacta, con acceso a USB para mantenimiento.	Protección.

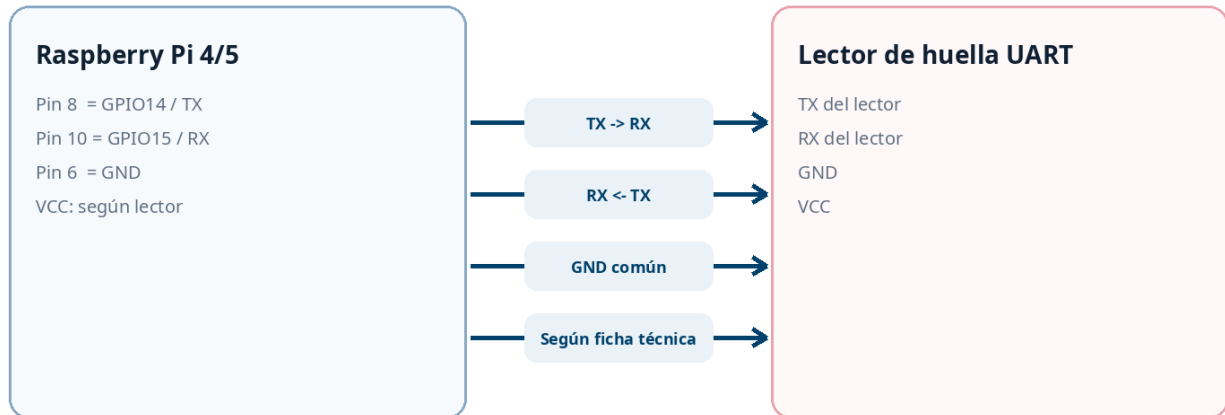
Antes de comprar el lector de huella

No todos los módulos usan exactamente el mismo voltaje de alimentación o niveles lógicos. Compra un modelo con documentación clara de VCC y UART. Nunca conectes una salida TTL de 5 V directamente a una entrada de 3.3 V sin verificar compatibilidad o usar adaptación de nivel.

3. Ensamblaje con Raspberry Pi

El código V85 espera el lector en el UART del sistema, por defecto /dev/serial0 a 57600 baudios. En una Raspberry Pi con cabecera estándar, la conexión de UART se hace usando GPIO14/TX y GPIO15/RX. El TX de un equipo siempre debe ir al RX del otro.

Cableado recomendado - Raspberry Pi



Siempre comparte GND. Verifica el voltaje exacto del lector antes de conectar VCC o señales.

Tabla de conexión

Lector de huella	Raspberry Pi	Pin físico	Nota
TX	GPIO15 / RX	10	Datos desde el lector hacia la Raspberry.
RX	GPIO14 / TX	8	Comandos desde la Raspberry hacia el lector.
GND	GND	6	Tierra común obligatoria.
VCC	Según ficha técnica	Según diseño	No asumir 3.3 V o 5 V sin revisar el modelo.

Montaje físico recomendado

1. Fija la Raspberry y la fuente dentro de una carcasa ventilada, evitando que el usuario pueda tocar las conexiones.
2. Coloca el sensor a una altura cómoda y con el cristal ligeramente inclinado para reducir polvo y reflejos.
3. Conecta primero GND, después TX/RX y por último alimentación. Trabaja con el equipo apagado.
4. Si usarás pantalla táctil, conecta HDMI/DSI y USB/táctil según el modelo de pantalla. La pantalla no comparte el UART del lector.
5. Deja acceso de mantenimiento a microSD, USB y alimentación, pero evita que queden expuestos al usuario final.

Importante

Si utilizas un HAT u otro accesorio que ya ocupe GPIO14/GPIO15, usa un adaptador USB-UART para el sensor y cambia serial_port en config.json, por ejemplo a /dev/ttyUSB0.

4. Preparar Raspberry Pi OS y el UART

6. Instala Raspberry Pi OS en la microSD con Raspberry Pi Imager y completa la conexión Wi-Fi.
7. Actualiza el sistema y confirma que la fecha/hora sean correctas.
8. Abre `sudo raspi-config` y entra a Interface Options > Serial Port.
9. Desactiva el login shell por serial y habilita el hardware del puerto serial.
10. Reinicia la Raspberry y confirma que exista `/dev/serial0`.

```
sudo raspi-config
ls -l /dev/serial0
python3 --version
```

Por qué se desactiva la consola serial

El lector necesita el UART para enviar sus propios paquetes. Si Linux mantiene un shell sobre los mismos pines, los datos se mezclan y el sensor puede dejar de responder.

Copiar e instalar el software del checador

Copia la carpeta CHECADOR/RASPBERRY de este paquete V85 a la Raspberry. Desde esa carpeta ejecuta:

```
chmod +x instalar.sh
./instalar.sh
```

El instalador crea el usuario de servicio skilled, instala el entorno Python en `/opt/skilled-checador`, agrega permisos de dialout/video, instala dependencias y habilita `skilled-checador.service`.

Comprobar el servicio

```
sudo systemctl status skilled-checador
sudo journalctl -u skilled-checador -n 100 --no-pager
```

5. Registrar la terminal en RH

En el CRM entra al perfil de Recursos Humanos y abre Checador de asistencia. En el bloque Terminales físicas sincronizadas usa + Registrar terminal.

11. Captura nombre del terminal, ubicación y tipo Raspberry Pi.
12. Activa Huella y deja habilitado QR/número para contingencia.
13. Genera la credencial. El CRM muestra un código de terminal y un token una sola vez.
14. Copia ambos valores inmediatamente y guárdalos en la Raspberry. No publiques el token en capturas, correos o documentación externa.

Editar config.json

```
{
  "device_name": "Checador Acceso Principal",
  "device_code": "CODIGO_GENERADO_EN_RH",
  "device_token": "TOKEN_GENERADO_EN_RH",
  "sync_url": "https://...supabase.co/functions/v1/checador-sync",
  "timezone": "America/Mexico_City",
  "sync_continuous": true,
  "sync_interval_seconds": 300,
  "admin_pin": "CAMBIAR_POR_UN_PIN_INTERNO",
  "fingerprint": {
    "enabled": true,
    "serial_port": "/dev/serial0",
    "baudrate": 57600
  }
}
```

Cambio obligatorio antes de producción

Cambia admin_pin. El valor de ejemplo del paquete no debe quedarse como contraseña real del equipo. Después de editar config.json reinicia el servicio.

```
sudo nano /opt/skilled-checador/config.json
sudo systemctl restart skilled-checador
```

6. Cómo registrar las huellas en Raspberry Pi

El sensor guarda la plantilla biométrica dentro de su propia memoria. La base local del checador guarda únicamente la relación posición del sensor -> número de empleado. Las checadadas sincronizadas llevan la referencia técnica, no una imagen cruda de la huella.

Antes de enrolar

- El trabajador debe existir en RH y tener un número de empleado estable.
- `fingerprint.enabled` debe estar en `true` y el UART debe funcionar.
- Detén temporalmente el servicio para que no compita por `/dev/serial0` durante el enrolamiento.

```
sudo systemctl stop skilled-checador
```

Registrar una huella

Ejemplo: asociar el empleado 00125 a la posición 1 del lector:

```
sudo -u skilled /opt/skilled-checador/.venv/bin/python /opt/skilled-checador/enroll_fingerprint.py
00125 1
```

15. El sistema pide colocar el dedo. Mantén el dedo quieto hasta completar la primera lectura.
16. Retira completamente el dedo cuando lo indique.
17. Coloca el mismo dedo una segunda vez. Las dos lecturas deben coincidir.
18. La plantilla se guarda en la posición indicada del sensor y la asociación se guarda en `checador.db`.
19. Si la posición ya existe, el script pregunta si quieres reemplazarla.

Organización sugerida de posiciones

Usa posiciones consecutivas y lleva la relación desde el propio comando de listado. El flujo V85 usa posiciones 1 a 127. Un mismo trabajador puede tener más de un dedo registrado si se asigna otra posición al mismo número de empleado.

Listar las huellas registradas

```
sudo -u skilled /opt/skilled-checador/.venv/bin/python /opt/skilled-checador/list_fingerprints.py
```

Eliminar una huella

```
sudo -u skilled /opt/skilled-checador/.venv/bin/python /opt/skilled-checador/delete_fingerprint.py 1
```

Al eliminar, se borra la plantilla de esa posición del sensor y se desactiva la asociación local.

Reactivar el checador

```
sudo systemctl start skilled-checador
sudo systemctl status skilled-checador
```


7. Prueba de una checada por huella

20. Con el servicio activo, coloca una huella registrada.
21. El lector realiza búsqueda local y obtiene la posición de la plantilla.
22. El checador resuelve esa posición al número de empleado en SQLite.
23. Si es la primera checada del día, el modo automático la registra como entrada. Si la última fue entrada, registra salida.
24. Existe una protección de 10 segundos para evitar doble lectura accidental del mismo trabajador.
25. Con Internet, el evento se envía en el siguiente ciclo de sincronización. Sin Internet, queda pendiente localmente.

Ver estado local

La aplicación escucha de forma predeterminada en 127.0.0.1:8787. En la misma Raspberry puede abrirse:

```
http://127.0.0.1:8787
```

La pantalla muestra eventos recientes y pendientes. También existe /api/status para diagnóstico local.

Pruebas mínimas antes de ponerlo en recepción

Prueba	Resultado esperado
Entrada por huella	Se identifica empleado y se genera entrada.
Salida por huella	Mismo día, después de entrada, genera salida.
Doble lectura inmediata	Se rechaza/ignora por protección de segundos.
Huella no registrada	No crea checada de otro trabajador.
Wi-Fi desconectado	La checada queda local como pendiente.
Reconectar Wi-Fi	Los pendientes se sincronizan sin volver a capturarlos.
Reinicio eléctrico	La base local conserva eventos pendientes.

8. Operación sin Internet y sincronización

Raspberry Pi usa checador.db con SQLite en `journal_mode=WAL`. Cada evento tiene UUID, empleado, tipo, fecha/hora, método, referencia biométrica, estado de sincronización y error si existiera.

- `sync_continuous=true` activa la revisión periódica.
- `sync_interval_seconds=300` significa intento incremental aproximadamente cada 5 minutos.
- Los eventos aceptados pasan a `synced=1`; los errores permanecen pendientes para reintento/diagnóstico.
- La hora se registra con la zona America/Mexico_City configurada en el equipo.

Regla práctica

El checador debe seguir aceptando huellas aunque el enlace a Internet esté caído. La prioridad es no perder la hora real de entrada/salida; la nube se actualiza después.

Cierre semanal

El servicio V85 también mantiene una rutina de cierre semanal. Guarda en SQLite qué semanas ya se cerraron para no repetirlas y recupera cierres pendientes si el equipo estuvo apagado.

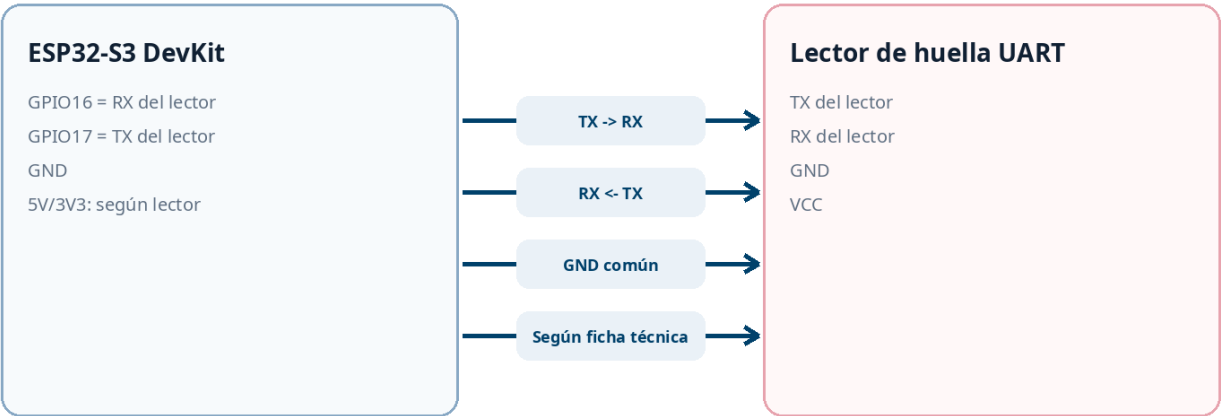
Relación con Nómina RH

Las checadas sincronizadas alimentan la vista semanal de RH, donde se visualizan por trabajador los días, entrada, salida, horas del día, faltantes de checada e incidencias. La nómina aplica después las reglas de cada esquema de pago.

9. Ensamblaje alternativo con ESP32-S3

El firmware incluido en CHECADOR/ESP32 está preparado para una ESP32-S3 con Arduino/PlatformIO. Usa Wi-Fi, LittleFS, Preferences, ArduinoJson y Adafruit Fingerprint. El UART del lector está configurado actualmente en GPIO16 y GPIO17.

Cableado actual - ESP32-S3



Siempre comparte GND. Verifica el voltaje exacto del lector antes de conectar VCC o señales.

Conexión usada por el firmware V85

Lector	ESP32-S3	Uso
TX	GPIO16	RX del UART del sensor.
RX	GPIO17	TX del UART del sensor.
GND	GND	Referencia común.
VCC	Según el módulo	Verifica la ficha técnica del sensor.

ESP32-S3 permite rutear señales UART mediante su matriz GPIO; si cambias pines, debes cambiar también `FingerSerial.begin(...)` en `src/main.cpp`.

Configurar antes de compilar

En `src/main.cpp` reemplaza los valores de red y credencial del terminal: `WIFI_SSID`, `WIFI_PASSWORD`, `SYNC_URL`, `DEVICE_CODE` y `DEVICE_TOKEN`.

Compilar y cargar con PlatformIO

```
cd CHECADOR/ESP32
pio run
pio run -t upload
pio device monitor -b 115200
```

10. Registrar huellas en ESP32-S3

El firmware V85 ofrece comandos por el monitor serie. La plantilla queda en el lector y Preferences guarda la relación ID -> número de empleado.

Registrar

```
ENROLL 1 00125
```

El ESP32 pedirá dos capturas del mismo dedo a través del flujo de la librería. Al finalizar, la posición 1 queda vinculada al empleado 00125.

Asociar una posición ya enrolada

```
MAP 1 00125
```

Eliminar posición y asociación

```
UNMAP 1
```

Cuándo elegir ESP32

Es una buena opción si el equipo final solo necesita huella/código, LED/buzzer y sincronización. Si necesitas una UI táctil rica, diagnóstico visual, administración local o reconocimiento facial, la Raspberry Pi es más conveniente.

11. Seguridad y datos biométricos

- No guardes fotografías de huellas en Supabase ni en la interfaz del CRM. El diseño actual trabaja con plantilla dentro del sensor y referencia local.
- El código y token de terminal deben tratarse como credenciales. Si un equipo se pierde, rota su token desde RH.
- Cambia el PIN administrativo del archivo config.json antes de producción.
- Protege físicamente USB, microSD y cableado. El usuario normal solo debe tener acceso al sensor y a la pantalla de checada.
- Mantén Wi-Fi de la terminal en una red controlada. Si la infraestructura lo permite, separa estos dispositivos de redes de invitados.
- No expongas el puerto local 8787 hacia Internet. En la configuración incluida está ligado a 127.0.0.1.

Consentimiento y política interna

Antes de usar biometría con personal real, la empresa debe definir su procedimiento interno de autorización, aviso y conservación conforme a las obligaciones aplicables. El manual técnico describe el funcionamiento del equipo; no sustituye asesoría legal de privacidad.

Respaldo

En Raspberry Pi, el archivo crítico local es /opt/skilled-checador/checador.db. Haz respaldos únicamente con acceso administrativo y protege el archivo porque contiene eventos y la relación entre referencias biométricas y números de empleado.

12. Solución de problemas

Síntoma	Revisión	Acción
El sensor no responde	UART ocupado, TX/RX invertidos, baudrate, alimentación.	Detén el servicio, revisa /dev/serial0 y cruza TX/RX.
No reconoce ninguna huella	Sensor sin plantillas o mapeo local ausente.	Ejecuta list_fingerprints.py y vuelve a enrolar.
Reconoce ID pero no empleado	Falta asociación en SQLite.	Reemplaza/enrola la posición con el número correcto.
Funciona local pero no sube	Wi-Fi, DNS, sync_url, device_code/token.	Verifica red y credencial generada en RH.
Muchos eventos pendientes	Backend no acepta o no hay Internet.	Revisa journalctl, fecha/hora y token de terminal.
Doble checada	Usuario vuelve a poner el dedo después del bloqueo.	Mantén el dedo hasta confirmación y retíralo; el software incluye cooldown.
Hora incorrecta	Zona horaria o reloj del equipo.	Usa America/Mexico_City y confirma sincronización de hora.
ESP32 no conecta al sensor	Pines, baudrate o alimentación.	Revisa GPIO16/17 y prueba intercambio TX/RX.

Comandos de diagnóstico Raspberry

```
sudo systemctl status skilled-checador
sudo journalctl -u skilled-checador -f
ls -l /dev/serial0
ip addr
python3 -c "import sqlite3; print(sqlite3.connect('/opt/skilled-checador/checador.db').execute('select count(*) from events where synced=0').fetchone())"
```

13. Checklist de puesta en marcha

- ☐ Raspberry/ESP32 fijada y protegida físicamente.
- ☐ Fuente estable y cableado sin terminales expuestas.
- ☐ GND común entre lector y controlador.
- ☐ Voltaje del lector verificado con su ficha técnica.
- ☐ UART probado antes de cerrar la carcasa.
- ☐ Terminal creada en RH y token copiado de forma segura.
- ☐ PIN administrativo cambiado.
- ☐ Huella de prueba registrada y listada correctamente.
- ☐ Entrada y salida probadas con Internet.
- ☐ Entrada y salida probadas sin Internet.
- ☐ Reconexión probada y cola pendiente sincronizada.
- ☐ Vista semanal de RH confirma horas por trabajador.
- ☐ Reinicio eléctrico probado sin pérdida de eventos.
- ☐ Respaldo y procedimiento de mantenimiento definidos.

Criterio para liberar el piloto

No poner el checador como única fuente de asistencia hasta completar por lo menos una semana de piloto con comparación manual. Cuando los registros coincidan, se puede retirar progresivamente la doble captura.

Estructura de archivos entregada en V85

Ruta	Uso
CHECADOR/RASPBERRY/app.py	Servidor local, checadas, sync y lazo de huella.
CHECADOR/RASPBERRY/store.py	SQLite, eventos y mapeo biométrico.
CHECADOR/RASPBERRY/enroll_fingerprint.py	Alta de huella.
CHECADOR/RASPBERRY/list_fingerprints.py	Revisión de posiciones y mapeos.
CHECADOR/RASPBERRY/delete_fingerprint.py	Baja de plantilla y mapeo.
CHECADOR/RASPBERRY/config.json	Credencial, red lógica, UART y parámetros.
CHECADOR/ESP32/src/main.cpp	Firmware ESP32-S3.
supabase/functions/checador-sync/index.ts	Recepción segura de eventos en la nube.

14. Fuentes técnicas oficiales consultadas

Estas referencias sirven para confirmar el uso de UART, configuración de puerto serial, matriz GPIO y el flujo de enrolamiento del lector. El cableado final debe respetar siempre la ficha técnica del modelo de sensor comprado.

Raspberry Pi Documentation - Configuration / Serial Port

<https://www.raspberrypi.com/documentation/computers/configuration.html>

Raspberry Pi Documentation - Raspberry Pi computer hardware

<https://www.raspberrypi.com/documentation/computers/raspberry-pi.html>

Espressif ESP32-S3 - UART

<https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/peripherals/uart.html>

Espressif ESP32-S3 - GPIO Matrix

<https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/peripherals/gpio.html>

Adafruit Optical Fingerprint Sensor - Python/CircuitPython

<https://learn.adafruit.com/adafruit-optical-fingerprint-sensor/circuitpython>

Decisión recomendada para el primer piloto

Raspberry Pi + pantalla + lector UART de huella + código de trabajador como contingencia. Una vez estabilizado el piloto, el mismo backend permite incorporar terminales ESP32-S3 en accesos secundarios.

Fin del manual • Skilled CRM V85